Lifespan Events

You can define logic (code) that should be executed before the application **starts up**. This means that this code will be executed **once**, **before** the application **starts receiving requests**.

The same way, you can define logic (code) that should be executed when the application is **shutting down**. In this case, this code will be executed **once**, **after** having handled possibly **many requests**.

Because this code is executed before the application **starts** taking requests, and right after it **finishes** handling requests, it covers the whole application **lifespan** (the word "lifespan" will be important in a second 😉).

This can be very useful for setting up **resources** that you need to use for the whole app, and that are **shared** among requests, and/or that you need to **clean up** afterwards. For example, a database connection pool, or loading a shared machine learning model.

Use Case

Let's start with an example **use case** and then see how to solve it with this.

Let's imagine that you have some **machine learning models** that you want to use to handle requests. 🤖

The same models are shared among requests, so, it's not one model per request, or one per user or something similar.

Let's imagine that loading the model can **take quite some time**, because it has to read a lot of **data from disk**. So you don't want to do it for every request.

You could load it at the top level of the module/file, but that would also mean that it would **load the model** even if you are just running a simple automated test, then that test would be **slow** because it would have to wait for the model to load before being able to run an independent part of the code.

That's what we'll solve, let's load the model before the requests are handled, but only right before the application starts receiving requests, not while the code is being loaded.

Lifespan

You can define this *startup* and *shutdown* logic using the `lifespan` parameter of the `FastAPI` app, and a "context manager" (I'll show you what that is in a second).

Let's start with an example and then see it in detail.

We create an async function `lifespan()` with `yield` like this:

Python 3.10+

```python
from contextlib import asynccontextmanager

from fastapi import FastAPI


def fake_answer_to_everything_ml_model(x: float):
    return x * 42
```

```python
ml_models = {}


@asynccontextmanager
async def lifespan(app: FastAPI):
    # Load the ML model
    ml_models["answer_to_everything"] = fake_answer_to_everything_ml_model
    yield
    # Clean up the ML models and release the resources
    ml_models.clear()


app = FastAPI(lifespan=lifespan)


@app.get("/predict")
async def predict(x: float):
    result = ml_models["answer_to_everything"](x)
    return {"result": result}
```

Here we are simulating the expensive *startup* operation of loading the model by putting the (fake) model function in the dictionary with machine learning models before the `yield`. This code will be executed **before** the application **starts taking requests**, during the *startup*.

And then, right after the `yield`, we unload the model. This code will be executed **after** the application **finishes handling requests**, right before the *shutdown*. This could, for example, release resources like memory or a GPU.

Tip

The `shutdown` would happen when you are **stopping** the application.

Maybe you need to start a new version, or you just got tired of running it. 🤷

Lifespan function

The first thing to notice, is that we are defining an async function with `yield`. This is very similar to Dependencies with `yield`.

Python 3.10+

```python
from contextlib import asynccontextmanager

from fastapi import FastAPI


def fake_answer_to_everything_ml_model(x: float):
    return x * 42


ml_models = {}


@asynccontextmanager
async def lifespan(app: FastAPI):
    # Load the ML model
    ml_models["answer_to_everything"] = fake_answer_to_everything_ml_model
    yield
    # Clean up the ML models and release the resources
    ml_models.clear()


app = FastAPI(lifespan=lifespan)


@app.get("/predict")
```

```
async def predict(x: float):
    result = ml_models["answer_to_everything"](x)
    return {"result": result}
```

The first part of the function, before the `yield`, will be executed **before** the application starts.

And the part after the `yield` will be executed **after** the application has finished.

Async Context Manager

If you check, the function is decorated with an `@asynccontextmanager`.

That converts the function into something called an "**async context manager**".

Python 3.10+

```
from contextlib import asynccontextmanager

from fastapi import FastAPI


def fake_answer_to_everything_ml_model(x: float):
    return x * 42


ml_models = {}


@asynccontextmanager
async def lifespan(app: FastAPI):
    # Load the ML model
    ml_models["answer_to_everything"] = fake_answer_to_everything_ml_model
    yield
    # Clean up the ML models and release the resources
    ml_models.clear()


app = FastAPI(lifespan=lifespan)


@app.get("/predict")
async def predict(x: float):
    result = ml_models["answer_to_everything"](x)
    return {"result": result}
```

A **context manager** in Python is something that you can use in a `with` statement, for example, `open()` can be used as a context manager:

```
with open("file.txt") as file:
    file.read()
```

In recent versions of Python, there's also an **async context manager**. You would use it with `async with`:

```
async with lifespan(app):
    await do_stuff()
```

When you create a context manager or an async context manager like above, what it does is that, before entering the `with` block, it will execute the code before the `yield`, and after exiting the `with` block, it will execute the code after the `yield`.

In our code example above, we don't use it directly, but we pass it to FastAPI for it to use it.

The `lifespan` parameter of the `FastAPI` app takes an **async context manager**, so we can pass our new `lifespan` async context manager to it.

Python 3.10+

```python
from contextlib import asynccontextmanager

from fastapi import FastAPI


def fake_answer_to_everything_ml_model(x: float):
    return x * 42


ml_models = {}


@asynccontextmanager
async def lifespan(app: FastAPI):
    # Load the ML model
    ml_models["answer_to_everything"] = fake_answer_to_everything_ml_model
    yield
    # Clean up the ML models and release the resources
    ml_models.clear()


app = FastAPI(lifespan=lifespan)


@app.get("/predict")
async def predict(x: float):
    result = ml_models["answer_to_everything"](x)
    return {"result": result}
```

Alternative Events (deprecated)

Warning

The recommended way to handle the *startup* and *shutdown* is using the `lifespan` parameter of the `FastAPI` app as described above. If you provide a `lifespan` parameter, `startup` and `shutdown` event handlers will no longer be called. It's all `lifespan` or all events, not both.

You can probably skip this part.

There's an alternative way to define this logic to be executed during *startup* and during *shutdown*.

You can define event handlers (functions) that need to be executed before the application starts up, or when the application is shutting down.

These functions can be declared with `async def` or normal `def`.

`startup` event

To add a function that should be run before the application starts, declare it with the event `"startup"`:

Python 3.10+

```python
from fastapi import FastAPI

app = FastAPI()
```

```
items = {}


@app.on_event("startup")
async def startup_event():
    items["foo"] = {"name": "Fighters"}
    items["bar"] = {"name": "Tenders"}


@app.get("/items/{item_id}")
async def read_items(item_id: str):
    return items[item_id]
```

In this case, the `startup` event handler function will initialize the items "database" (just a `dict`) with some values.

You can add more than one event handler function.

And your application won't start receiving requests until all the `startup` event handlers have completed.

`shutdown` event

To add a function that should be run when the application is shutting down, declare it with the event `"shutdown"`:

Python 3.10+

```
from fastapi import FastAPI

app = FastAPI()


@app.on_event("shutdown")
def shutdown_event():
    with open("log.txt", mode="a") as log:
        log.write("Application shutdown")


@app.get("/items/")
async def read_items():
    return [{"name": "Foo"}]
```

Here, the `shutdown` event handler function will write a text line `"Application shutdown"` to a file `log.txt`.

Info

In the `open()` function, the `mode="a"` means "append", so, the line will be added after whatever is on that file, without overwriting the previous contents.

Tip

Notice that in this case we are using a standard Python `open()` function that interacts with a file.

So, it involves I/O (input/output), that requires "waiting" for things to be written to disk.

But `open()` doesn't use `async` and `await`.

So, we declare the event handler function with standard `def` instead of `async def`.

`startup` and `shutdown` together

There's a high chance that the logic for your *startup* and *shutdown* is connected, you might want to start something and then finish it, acquire a resource and then release it, etc.

Doing that in separated functions that don't share logic or variables together is more difficult as you would need to store values in global variables or similar tricks.

Because of that, it's now recommended to instead use the `lifespan` as explained above.

Technical Details

Just a technical detail for the curious nerds. 🤓

Underneath, in the ASGI technical specification, this is part of the Lifespan Protocol, and it defines events called `startup` and `shutdown`.

> **Info**
>
> You can read more about the Starlette `lifespan` handlers in Starlette's Lifespan' docs.
>
> Including how to handle lifespan state that can be used in other areas of your code.

Sub Applications

🚨 Keep in mind that these lifespan events (startup and shutdown) will only be executed for the main application, not for Sub Applications - Mounts.